

1h 9m left

Language

1. Code Question 1



ALL



1

2

Amazon Academy has launched a new course on Quantum Physics.

The course has n chapters, each chapter has $memory[i]$ memory points, which a student gains or loses while reading that chapter. The course has a requirement: in order to study the i^{th} chapter, the student must revisit all the previous chapters. A student gains $memory[0] + memory[1] + \dots + memory[i]$ memory points for reading the i^{th} chapter. The *total memory points* is the sum of memory points gained while reading each chapter.

Students can read the chapters in any order, and want to maximize their *total memory points*. Find the maximum *total memory points* a student can score ensuring that all the chapters are read.

Example

$memory = [3, 4, 5]$

Considering all permutations of reading the chapters and their total memory points:

- $[3, 4, 5] \rightarrow$ Corresponding memory points will be $[3, 7, 12]$, thus total memory points will be $3 + 7 + 12 = 21$

Test R

memory points gained while reading each chapter.

Students can read the chapters in any order, and want to maximize their *total memory points*. Find the maximum *total memory points* a student can score ensuring that all the chapters are read.

Example

$memory = [3, 4, 5]$

Considering all permutations of reading the chapters and their total memory points:

- $[3, 4, 5] \rightarrow$ Corresponding memory points will be $[3, 7, 12]$, thus total memory points will be $3 + 7 + 12 = 21$
- $[3, 5, 4] \rightarrow$ Corresponding memory points will be $[3, 8, 12]$, thus total memory points will be $3 + 8 + 12 = 23$
- $[4, 3, 5] \rightarrow$ Corresponding memory points will be $[4, 7, 12]$, thus total memory points will be $4 + 7 + 12 = 23$
- $[4, 5, 3] \rightarrow$ Corresponding memory points will be $[4, 9, 12]$, thus total memory points will be $4 + 9 + 12 = 25$
- $[5, 3, 4] \rightarrow$ Corresponding memory points will be $[5, 8, 12]$, thus total memory points will be $5 + 8 + 12 = 25$
- $[5, 4, 3] \rightarrow$ Corresponding memory points will be $[5, 9, 12]$, thus total memory points will be $5 + 9 + 12 = 26$

The maximum number of total memory points is

Test R

1h 5m left

Returns

long_int: the maximum possible total memory points

Constraints

- $1 \leq n \leq 10^5$
- $-10^3 \leq \text{memory}[i] \leq 10^3$

► Input Format For Custom Testing

▼ Sample Case 0

Sample Input For Custom Testing

STDIN	FUNCTION
3	→ memory[] size, n = 3
1	→ memory = [1, -2, 3]
-2	
3	

Sample Output

9

Explanation

The permutation with maximum total memory points is [3, 1, -2]. The corresponding memory points for each chapter are [3, 4, 2] resulting in the maximum sum of $3 + 4 + 2 = 9$.

► Sample Case 1

Language C

```
1 > #in
19
20 /*
21 *
22 *
23 *
24 *
25 */
26
27 long
28
29 }
30
31 > int m
```

Test Results

1h 8m left



ALL



1

2

To be efficient, Amazon must optimally distribute parcels among their delivery agents. Initially, there are n agents, and the number of parcels assigned to the i^{th} agent is $parcels[i]$. There are $extra_parcels$ parcels that also need to be shipped. The extra parcels should be assigned such that the maximum number of parcels assigned to any one agent is minimized.

Given an integer array, $parcels$, and an integer, $extra_parcels$, find the minimum possible value of the maximum number of parcels any agent must deliver.

Example

Suppose $n = 5$, $parcels = [7, 5, 1, 9, 1]$ and $extra_parcels = 25$.

An optimal assignment is shown.

Parcels	Extra Parcels	Total Parcels
7	3	10
5	4	9
1	9	10
9	1	10
1	8	9

There are other possible optimal assignments, but the minimum value of the maximum number of parcels any agent will deliver is 10.

Function Description

Complete the function `findMinimumMaximum` in the editor.

Test R

Search

1	8	9
---	---	---

1h 7m left



ALL



1

2

There are other possible optimal assignments, but the minimum value of the maximum number of parcels any agent will deliver is 10.

Function Description

Complete the function *findMinimumMaximum* in the editor below.

findMinimumMaximum takes the following arguments:

int parcels[n]: the initial number of parcels assigned to the delivery agents

long int extra_parcels: the extra parcels to be assigned

Returns

long int: the minimum possible value of the maximum number of parcels an agent must deliver

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq \text{extra_parcels} \leq 10^{15}$
- $1 \leq \text{parcels}[i] \leq 10^9$

Input Format For Custom Testing

The first line contains an integer, *n*, the number of elements in *parcels*.

Each of the next *n* lines contains an integer, *parcels[i]*.

The next line contains an integer, *extra_parcels*.

Test

Thank you for your application

HackerRank Question - Code

hackerrank.com/test/rfe1p4jtmt/questions/etld9noflm2

1h 7m left

There are other possible optimal assignments, but the minimum value of the maximum number of parcels any agent will deliver is 10.

ALL

Function Description

Complete the function *findMinimumMaximum* in the editor below.

findMinimumMaximum takes the following arguments:

int parcels[n]: the initial number of parcels assigned to the delivery agents

long int extra_parcels: the extra parcels to be assigned

Returns

long int: the minimum possible value of the maximum number of parcels an agent must deliver

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq \text{extra_parcels} \leq 10^{15}$
- $1 \leq \text{parcels}[i] \leq 10^9$

Input Format For Custom Testing

The first line contains an integer, *n*, the number of elements in *parcels*.

Each of the next *n* lines contains an integer, *parcels[i]*.

The next line contains an integer, *extra_parcels*.



Search

1h 7m left

▼ Sample Case 0

Sample Input For Custom Testing

STDIN		FUNCTION
3	→	parcels[] size n = 3
1	→	parcels = [1, 2, 3]
2		
3		
3	→	extra_parcels = 3

Sample Output

1 3

Explanation

It is optimal to assign 2 extra parcels to the first agent and 1 extra parcel to the second agent. Finally, each agent will have to deliver 3 parcels.

▼ Sample Case 1

Sample Input For Custom Testing

STDIN		FUNCTION
1	→	parcels[] size n = 1
1	→	parcels = [1]
3	→	extra_parcels = 3

Sample Output

4

Explanation

Since there is only one agent, all the extra parcels are assigned to the same.



Search

Tes

below.

`findMinimumMaximum` takes the following arguments:

`int parcels[n]`: the initial number of parcels assigned to delivery agents

`long int extra_parcels`: the extra parcels to be assigned

Returns

`long int`: the minimum possible value of the maximum number of parcels an agent must deliver

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq \text{extra_parcels} \leq 10^{15}$
- $1 \leq \text{parcels}[i] \leq 10^9$

Input Format For Custom Testing

The first line contains an integer, n , the number of elements in `parcels`.

Each of the next n lines contains an integer, `parcels[i]`.

The next line contains an integer, `extra_parcels`.

Sample Case 0

Sample Input For Custom Testing

STDIN

3

1

2

3

9

9

FUNCTION

`parcels[]` size $n = 3$
`parcels = [1, 2, 3]`

2. Code Question 2

Amazon has several warehouses that store piles of boxes containing goods to be shipped.

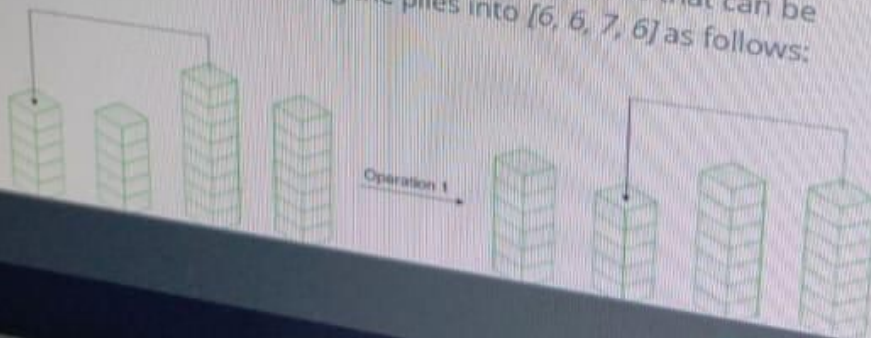
In one such warehouse, there are a total of n piles numbered $1, 2, \dots, n$, where the i^{th} pile has $\text{boxes}[i]$ number of boxes. To have an even distribution of boxes, the caretaker can do the following operation any number of times (possibly zero):

- Choose two distinct piles, i and j ($1 \leq i, j \leq n$), such that $\text{boxes}[i] > 0$.
- Remove one box from pile i and place it on pile j . More formally, increment $\text{boxes}[j]$ by 1 and decrement $\text{boxes}[i]$ by 1.

For even distribution, the caretaker wishes to minimize the difference between the maximum and the minimum number of boxes in the piles. Call the minimum difference achievable d . The goal is to find the minimum number of operations required to achieve the difference d .

Example

Consider the number of piles to be $n = 4$ and the boxes in them are $\text{boxes} = [5, 5, 8, 7]$. The minimum possible difference that can be achieved is 1 by transforming the piles into $[6, 6, 7, 6]$ as follows:



54m left



ALL

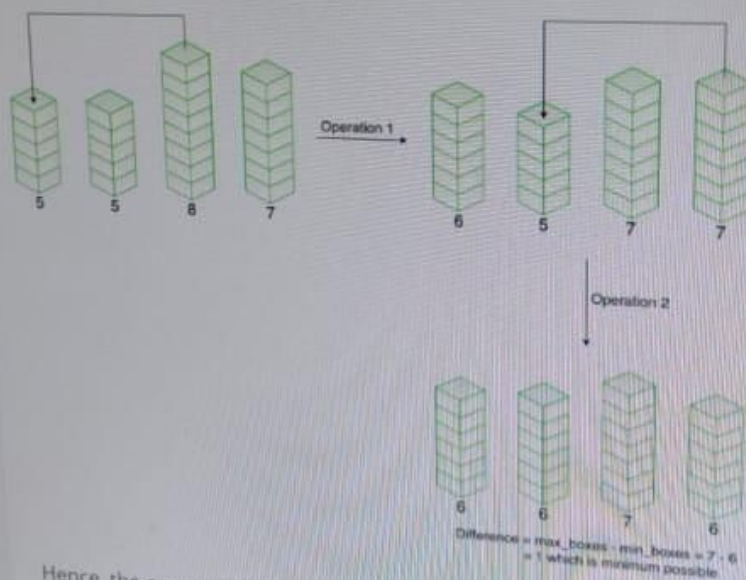


2

Consider the number of piles to be $n = 4$ and the boxes in them are $boxes = [5, 5, 8, 7]$. The minimum possible difference that can be achieved is 1 by transforming the piles into $[6, 6, 7, 6]$ as follows:

Example

Consider the number of piles to be $n = 4$ and the boxes in them are $boxes = [5, 5, 8, 7]$. The minimum possible difference that can be achieved is 1 by transforming the piles into $[6, 6, 7, 6]$ as follows:



Hence, the answer is 2. Note that there are multiple optimal pile configurations such as $[6, 6, 7, 6]$, $[6, 6, 6, 7]$.

Function Description

Complete the function `getFewestOperations` in the editor below.

`getFewestOperations` has the following parameter:

- `int boxes[n]`: the number of boxes in each pile

Language

```
1 > #
9
10 /*
11 *
12 *
13 *
14 *
15 */
16
17 long
18
19
20
21 > int
```

Test Results

Search

ZEBS

Thank you for your application

HackerRank Question - Code

hackerrank.com/test/rfe1p4jmt/questions/2al6nj34jf1

54m left

Hence, the answer is 2. Note that there are multiple optimal pile configurations such as [6, 6, 7, 6], [6, 6, 6, 7].

Function Description

Complete the function `getFewestOperations` in the editor below.

`getFewestOperations` has the following parameter:

`int boxes[n]`: the number of boxes in each pile

Returns

`long_int`: the minimum number of operations to achieve the minimum possible difference

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq boxes[i] \leq 10^9$

Input Format For Custom Testing

Sample Case 0

Sample Input For Custom Testing

STDIN	FUNCTION
3	→
2	→ boxes[] size n = 3
4	→ boxes = [2, 4, 1]
1	

Sample Output

1

Test Resu

Search

ZEB

54m left

Sample Input For Custom Testing

STDIN	FUNCTION
-------	----------

Text selection has been disabled in this region

ALL

1

Sample Output

1

Explanation

Move a box from pile 2 to pile 3: [2, 4, 1] → [2, 3, 2].

▼ Sample Case 1

Sample Input For Custom Testing

STDIN	FUNCTION
5	
4	→ boxes[] size n = 5
4	→ boxes = [4, 4, 4, 4, 4]
4	
4	
4	

Sample Output

0

Explanation

All of the piles are equal already.



Search

Test Result



52m left

2. Code Question 2

Amazon games have recently launched a new multi-player tournament on the platform. Each game of the tournament has 3 rounds. The players are provided with exactly three power boosters at the start of the game and each player is aware of the power boosters of their opponent. In each round, the player can choose to compete with any of the power boosters and any power booster can be used at most once in a particular game. In any particular round, the player with a higher power booster wins.

A player X is considered to be capable of defeating player Y if there exists a rearrangement of power boosters of X such that some rearrangement of power boosters of Y can be defeated in at least 2 out of the three rounds. For example, if power boosters of $X = [9, 5, 11]$ and $Y = [7, 12, 3]$, then

- If Y uses the boosters in order $[3, 7, 12]$ then X can use it in order $[11, 9, 5]$ and X wins 2 out of the three games as $11 > 3$ and $9 > 7$. Thus X is capable of defeating Y.
- If X uses the boosters in order $[5, 9, 11]$ then Y can use it in order $[7, 12, 3]$ and Y wins 2 out of the three games as $7 > 5$ and $12 > 9$. Thus Y is also capable of defeating X.

Another example is $X = [1, 2, 3]$ and $Y = [3, 4, 5]$

Language C++20

```
1 > #include
9
10 /*
11 * Com
12 *
13 * The
14 * The
15 * 1.
16 * 2.
17 * 3.
18 */
19
20 int g
21
22 }
23
24 > int n
```

Test Result



52m left

- If X uses the boosters in order [5, 9, 11] then Y can use it in order [7, 12, 3] and Y wins 2 out of the three games as $7 > 5$ and $12 > 9$. Thus Y is also capable of defeating X.



ALL



2

Another example is $X = [1, 2, 3]$ and $Y = [3, 4, 5]$. Here X is not capable of defeating Y as any rearrangement of Y can not be defeated by X in at least two rounds.

Given the power boosters provided to n players where the three power boosters of the i^{th} player are defined by $(\text{booster_a}[i], \text{booster_b}[i], \text{booster_c}[i])$, find the number of players who are capable of defeating all other players in a game by using their power boosters optimally. It is guaranteed that all powers of each player's power boosters are distinct.

Example

Consider the number of players to be $n = 3$, and their first set of power boosters to be $\text{booster_a} = [9, 4, 2]$, their second set of power boosters to be $\text{booster_b} = [5, 12, 10]$ and their third set of power boosters to be $\text{booster_c} = [11, 3, 13]$. Thus, Player 1 has power boosters [9, 5, 11], Player 2 has [4, 12, 3] and Player 3 has power boosters [2, 10, 13].

- Player 1 can defeat Player 2 by playing in the following way: 5 against 3, 9 against 4, 11 against 12; Player 1 wins the first 2 rounds.
- Player 1 can defeat Player 3 by playing in the following way: 5 against 2, 9 against 13, 11 against 10; Player 1 wins the first and last rounds.

Language

1 > #

9

10 /

11

12

13

14

15

16

17

18

19

20

21

22

23

24 > i

Test Res

Thus, Player 1 and Player 3 can defeat all the players using their power boosters optimally. Thus, the answer is 2.

Function Description

Complete the function `getWinnerCount` in the editor below.

`getWinnerCount` has the following parameter:

`int booster_a[n]`: the first set of power boosters

`int booster_b[n]`: the second set of power boosters

`int booster_c[n]`: the third set of power boosters

Returns

`int`: the number of players capable of defeating all other players

Constraints

- $2 \leq n \leq 10^5$
- $1 \leq \text{booster_a}[i], \text{booster_b}[i], \text{booster_c}[i] \leq 10^9$, where $0 \leq i < n$
- All power boosters of each player are pairwise distinct.

► Input Format For Custom Testing

▼ Sample Case 0

Sample Input For Custom Testing

Test Res

51m left

► Input Format For Custom Testing

▼ Sample Case 0

Sample Input For Custom Testing

STDIN		FUNCTION
-----		-----
4	→	booster_a[] size n = 4
3	→	booster_a = [3, 4, 1,
16]		
4		
1		
16		
4	→	booster_b[] size n = 4
2	→	booster_b = [2, 11, 5,
6]		
11		
5		
6		
4	→	booster_c[] size n = 4
8	→	booster_c = [8, 7, 9,
10]		
7		
9		
10		

Sample Output

2

Explanation

Given $n = 4$, $\text{booster_a} = [3, 4, 1, 16]$, $\text{booster_b} = [2, 11, 5, 6]$, $\text{booster_c} = [8, 7, 9, 10]$.

Player 2 can defeat all players in the following way:

Test Re

2

Explanation

Given $n = 4$, $booster_a = [3, 4, 1, 16]$, $booster_b = [2, 11, 5, 6]$, $booster_c = [8, 7, 9, 10]$.

Player 2 can defeat all players in the following way:

- Against Player 1: 4 vs 2, 11 vs 8, 7 vs 3. Wins all 3 rounds
- Against Player 3: 4 vs 1, 11 vs 9, 7 vs 5. Wins all 3 rounds
- Against Player 4: 4 vs 16, 11 vs 10, 7 vs 6. Wins last 2 rounds

Player 4 is also capable of defeating all the players in the following way:

- Against Player 1: 16 vs 8, 6 vs 3, 10 vs 2. Wins all 3 rounds
- Against Player 2: 16 vs 11, 6 vs 4, 10 vs 11. Wins first 2 rounds
- Against Player 3: 16 vs 9, 6 vs 5, 10 vs 1. Wins all 3 rounds

Player 1 cannot defeat Player 2 and Player 4 using any combination of his power boosters. Similarly, Player 3 cannot defeat Player 4 using any combination of power boosters. Thus, the answer is 2.

► Sample Case 1

Test R

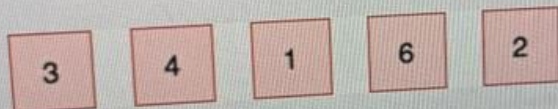


2. Code Question 2

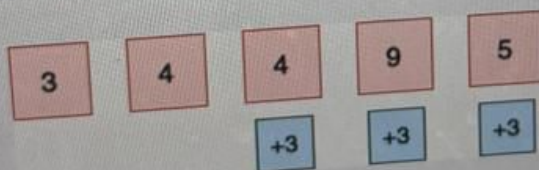
AWS provides scalable systems. A set of n servers are used for horizontally scaling an application. The goal is to have the computational power of the servers in non-decreasing order. To do so, you can increase the computational power of each server in any contiguous segment by x . Choose the values of x such that after the computational powers are in non-decreasing order, the sum of the x values is minimum.

Example

There are $n = 5$ servers and their computational power = $[3, 4, 1, 6, 2]$.

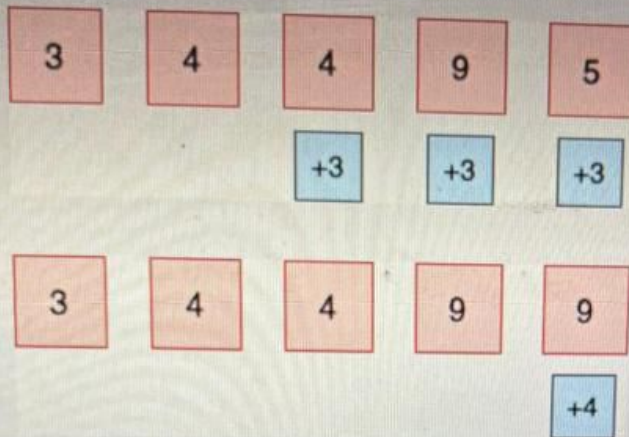


Add 3 units to the subarray (2, 4) and 4 units to the subarray (4, 4). The final arrangement of the servers is: $[3, 4, 4, 9, 9]$. The answer is $3 + 4 = 7$.



hackerrank.com/test/rfe1p4jtmt/questions/7g54ef1h03l

Add 3 units to the subarray (2, 4) and 4 units to the subarray (4, 4). The final arrangement of the servers is: [3, 4, 4, 9, 9]. The answer is $3 + 4 = 7$.



Function Description

Complete the function *findMinimumSum* in the editor below.

findMinimumSum has the following parameter(s):

int power[n]: the computational powers of *n* different servers

Returns

int: the minimum possible sum of integers required to make the array non-decreasing

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq \text{power}[i] \leq 10^9$

hackerrank.com/test/rfe1p4jtmt/questions/7g54ef1h03l

- $1 \leq n \leq 10^5$
- $1 \leq \text{power}[i] \leq 10^9$

▼ Input Format For Custom Testing

The first line contains an integer, n , denoting the number of elements in *power*.

Each line i of the n subsequent lines (where $0 \leq i < n$) contains an integer describing power_i .

▼ Sample Case 0

Sample Input For Custom Testing

STDIN	FUNCTION
-----	-----
3	→ power[] size n = 3
3	→ power = [3, 2, 1]
2	
1	

Sample Output

2

Explanation

Add 1 unit to subarray (1,2) and 1 unit to subarray (2, 2). The final arrangement of servers is [3, 3, 3].

2

1

Sample Output

2

Explanation

Add 1 unit to subarray (1,2) and 1 unit to subarray (2, 2). The final arrangement of servers is [3, 3, 3].

▼ Sample Case 1

Sample Input For Custom Testing

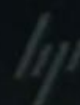
STDIN	FUNCTION
-----	-----
4	→ power[] size n = 4
3	→ power = [3, 5, 2, 3]
5	
2	
3	

Sample Output

3

Explanation

Add 3 units to subarray (2, 3). The final arrangement of servers is [3, 5, 5, 6].



1. Code Question 1

Amazon Shopping periodically has offers to attract more customers.

It recently launched an offer for n items in its inventory, where the i^{th} item offered reward points $points[i]$ to the customer purchasing the item. Every time an offer-bearing item is purchased, the customer gains the reward points associated with that item. Then the reward points of the remaining items are reduced by 1 unless it will reduce the points below 0.

Find the maximum possible reward points that can be gathered by purchasing the items optimally.

Note: Each item can be purchased at most once, in other words, $points[i]$ becomes 0 after the i^{th} item is purchased.

Example

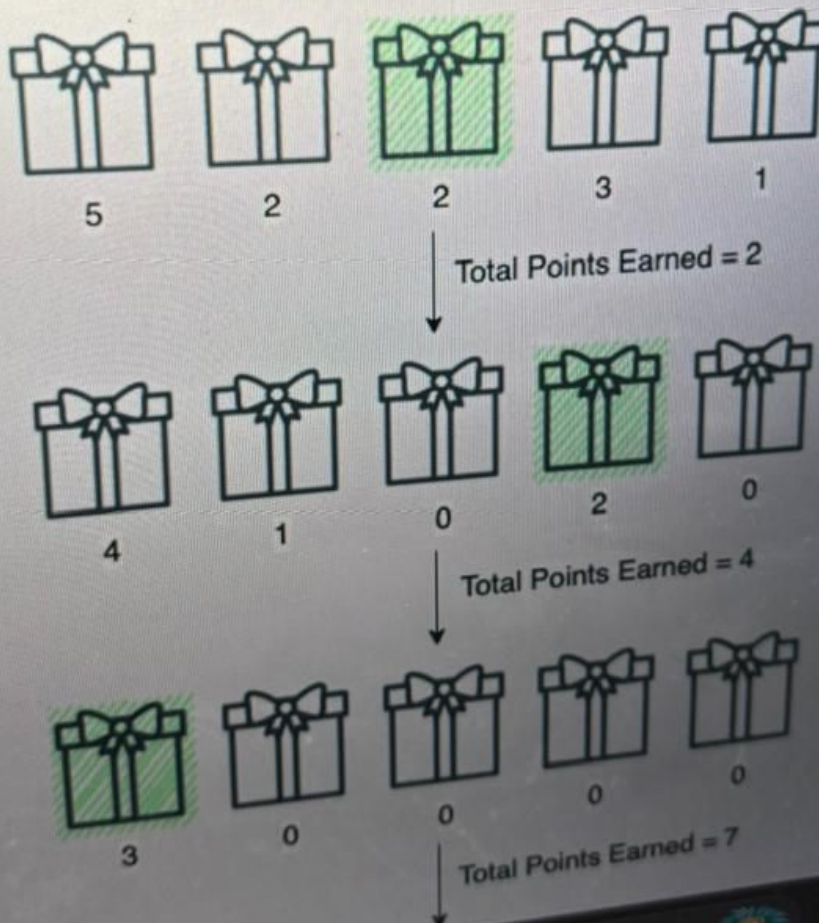
Consider the number of items to be $n = 5$, and their reward points to be $points = [5, 2, 2, 3, 1]$. The items can be purchased as follows:





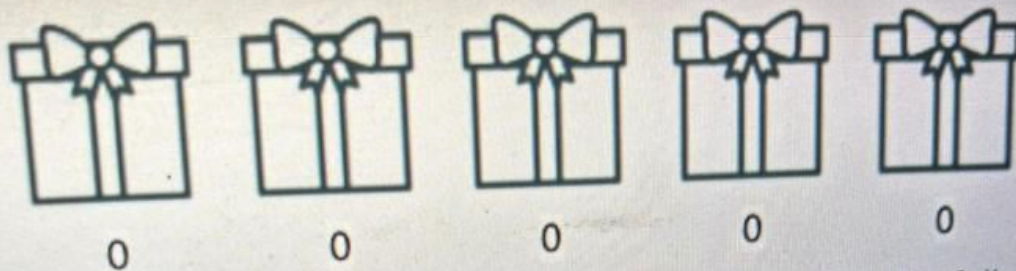
Example

Consider the number of items to be $n = 5$, and their reward points to be $points = [5, 2, 2, 3, 1]$. The items can be purchased as follows:



hackerrank.com/test/rfe1p4jtmt/questions/ct7kqk9ilft

Total Points Earned = 7



Considering 0-based indexing, the items can be purchased in the following order:

- First, purchase item 2, points earned = $points[2] = 2$. Points of remaining items after this purchase $points = [4, 1, 0, 2, 0]$.
- Next, purchase item 3, points earned = $points[3] = 2$. Points of remaining items after this purchase $points = [3, 0, 0, 0, 0]$.
- Finally, purchase item 0, points earned = $points[0] = 3$. Points of remaining items after this purchase $points = [0, 0, 0, 0, 0]$.

At this point, no items have reward points left. The maximum reward points is $2 + 2 + 3 = 7$.

Function Description

Complete the function `findMaxPoints` in the editor below.

`findMaxPoints` has the following parameter(s):
`int points[n]`: the reward points of each item

Returns

 hackerrank.com/test/rfe1p4jtmt/questions/ct7kqk9ilft`points[i].`

▼ Sample Case 0

Sample Input For Custom Testing

STDIN		FUNCTION
-----		-----
3	→	<code>points[]</code> size <code>n = 3</code>
5	→	<code>points = [5, 5, 5]</code>
5		
5		

Sample Output

12

Explanation

Using 0-based indexing, the items can be purchased in the following order:

- First, purchase item 0, points earned = `points[0] = 5`. Points of remaining items after this purchase `points = [0, 4, 4]`.
- Next, purchase item 1, points earned = `points[1] = 4` and `points = [0, 0, 3]`.
- Finally, purchase item 2, points earned = `points[2] = 3` and `points = [0, 0, 0]`.

The maximum reward points is $(5 + 4 + 3 = 12)$.

▶ Sample Case 1



Search

hackerrank.com/test/rfe1p4jtmt/questions/ct7kqk9ilft

- Next, purchase item 1, points earned = $points[1] = 4$ and $points = [0, 0, 5]$.
- Finally, purchase item 2, points earned = $points[2] = 3$ and $points = [0, 0, 0]$.

The maximum reward points is $(5 + 4 + 3 = 12)$.

▼ Sample Case 1

Sample Input For Custom Testing

STDIN	FUNCTION
5	$\rightarrow points[]$ size $n = 5$
1	$\rightarrow points = [1, 3, 5, 2, 4]$
3	
5	
2	
4	

Sample Output

9

Explanation

- First, purchase item 2, points earned = $points[2] = 5$. $points = [0, 2, 0, 1, 3]$.
- Next, purchase item 4, points earned = $points[4] = 3$. $points = [0, 1, 0, 0, 0]$.
- Finally, purchase item 1, points earned = $points[1] = 1$. $points = [0, 0, 0, 0, 0]$.

The maximum reward points is $5 + 3 + 1 = 9$.

  hackerrank.com/test/rfe1p4jtmt/questions/ct7kqk9ilft

Returns

long_int: the maximum reward points which can be collected

Constraints

- $1 \leq n \leq 10^5$
- $0 \leq \text{points}[i] \leq 10^6$

▼ Input Format For Custom Testing

The first line contains an integer, n , denoting the number of elements in *points*.

Each line i of the n subsequent lines (where $0 \leq i < n$) contains an integer, *points*[i].

▼ Sample Case 0

Sample Input For Custom Testing

STDIN		FUNCTION
-----		-----
3	→	<i>points</i> [] size $n = 3$
5	→	<i>points</i> = [5, 5, 5]
5		
5		

Sample Output



Search



1h 9m left



ALL



1

2

1. Code Question 1

An AWS client has brought servers and databases from data centers in different parts of the world for their application. For simplicity, let's assume all the servers and data centers are located on a 1-dimensional line.

You have been given the task of optimizing the network connection. Each data center must be connected to a server. The positions of n data centers and n servers are given in the form of arrays. Any particular data center, $center[i]$, can deliver to any particular server destination, $destination[j]$. The lag is defined distance between a data center at location x and a server destination at location y is $|x - y|$, i.e., the absolute difference between x and y . Determine the minimum lag to establish the entire network.

Example

There are $n = 3$ connections, the positions of data centers, $center = [1, 2, 2]$, and the positions of the server destinations, $destination = [5, 2, 4]$.

The most efficient deliveries are:

- The center at location 1 makes the first connection to the server at location 2.
- The center at location 2 makes the second connection to the server at location 4.
- The center at location 2 makes the third connection to the server at location 5.

Autocomplete Re

```
1 > #inclu
9 |
10 /*
11 * Comp
12 *
13 * The
LONG_IN
14 * The f
15 * 1. I
16 * 2. I
17 */
18
19 long getM
vector<in
20
21 }
22
23 > int main()
```

Test Results

1h 9m left



ALL



1

2

The minimum total lag is $= \text{abs}(1 - 2) + \text{abs}(2 - 4) + \text{abs}(2 - 5) = 1 + 2 + 3 = 6$.

Function Description

Complete the function `getMinDistance` in the editor below.

`getMinDistance` has the following parameter(s):

`int center[n]`: the positions of data centers

`int destination[n]`: the positions of server destinations

Returns

`long int`: the minimum lag

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq \text{center}[i], \text{destination}[i] \leq 10^9$

▼ Input Format For Custom Testing

The first line contains an integer, n , the number of elements in the array `center`.

Each line i of the n subsequent lines (where $0 \leq i < n$) contains an integer, `center[i]`.

The next line contains an integer, n , the number of elements in the array `destination`.

Language C++20

Autocomplete Rea

```
1 > #includ
9
10 /*
11  * Comp
12  *
13  * The f
14  * The fu
15  * 1. IN
16  * 2. IN
17  */
18
19 long getMi
vector<int
20
21 }
22
23 > int main()
```

▼ Sample Case 0

Sample Input For Custom Testing

STDIN	FUNCTION
5	→ center[] size n = 5
3	→ center = [3, 1, 6, 8, 9]
1	
6	
8	
9	
5	→ destination[] size n = 5
2	→ destination = [2, 3, 1, 7, 9]
3	
1	
7	
9	

Sample Output

5

Explanation

You may create the connections as:

center = [1, 3, 6, 8, 9]

destination = [1, 2, 3, 7, 9]

Minimum total distance = $\text{abs}(1 - 1) + \text{abs}(2 - 3) + \text{abs}(3 - 6) + \text{abs}(7 - 8) + \text{abs}(9 - 9) = 0 + 1 + 3 + 1 + 0 = 5$

Explanation

You may create the connections as:

center = [1, 3, 6, 8, 9]

destination = [1, 2, 3, 7, 9]

Minimum total distance = $\text{abs}(1 - 1) + \text{abs}(2 - 3) + \text{abs}(3 - 6) + \text{abs}(7 - 8) + \text{abs}(9 - 9) = 0 + 1 + 3 + 1 + 0 = 5$

▼ Sample Case 1

Sample Input For Custom Testing

STDIN	FUNCTION
-----	-----
2	→ center[] size n = 2
4	→ center = [4, 6]
6	
2	→ destination[] size n = 2
5	→ destination = [5, 5]
5	

Sample Output

2

Explanation

You may create the connections as:

center = [4, 6]

destination = [5, 5]

Minimum total distance = $\text{abs}(5 - 4) + \text{abs}(5 - 6) = 1 + 1 = 2$